

PrepLab — Session 3 Notes

Network Security Controls — Visibility Assessment

Objective

PrepLab has no traditional security stack (no firewall, IDS/IPS, or SIEM). Its only "controls" are the Docker network split (frontend/backend) and whatever each container writes to its own logs. The goal of this session is to determine, empirically rather than by assumption, what these controls can and cannot see — and to identify the two most significant visibility gaps.

Step 1 — Confirm the Network Topology

Inspected both Docker networks to confirm they match the design in docker-compose.yml:

```
$ docker network ls
NETWORK ID      NAME                                DRIVER  SCOPE
e4143e5bbd52    lab-20260827_backend              bridge  local
5922b31e6b72    lab-20260827_frontend             bridge  local
```

Network	Subnet	Containers attached
frontend	172.18.0.0/16	web (172.18.0.3), files (172.18.0.2)
backend	172.19.0.0/16	web (172.19.0.3), db (172.19.0.2)

Result: topology matches the intended design. **files** has no route to **db** at all — different subnets, no shared network. **web** is the only container bridging both.

Step 2 — Test the Boundary Instead of Trusting the Diagram

The Docker network view above only shows the internal picture. docker-compose.yml also publishes the database port straight to the host:

```
db:
  ports:
    - "5432:5432"
```

This is a route that exists entirely outside the frontend/backend network diagram. Tested directly from the host (bypassing Docker networking altogether):

```
$ (echo > /dev/tcp/localhost/5432) 2>&1 && echo "PORT OPEN" || echo "PORT CLOSED/FILTERED"
PORT OPEN
```

```
$ curl -v telnet://localhost:5432
* Trying [::1]:5432...
* Established connection to localhost (::1 port 5432) from ::1 port 55332
```

CONFIRMED

The database is directly reachable from the host, completely bypassing the intended frontend/backend segmentation. This is not a hidden "backdoor" — it is a plainly declared, if careless, docker-compose misconfiguration (planted weakness #8). In a properly secured deployment the **ports:** mapping for db would be removed entirely, since web reaches db automatically over the backend network without it.

Step 3 — Does Anything See That Connection?

Having proven the port is reachable, the next question is whether anything would notice if it were used. Checked the database container's own log output immediately after the connection tests above:

```
$ docker compose logs db --tail 50
...
2026-09-14 10:33:46.594 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2026-09-14 10:33:46.594 UTC [1] LOG:  listening on IPv6 address ":::", port 5432
2026-09-14 10:33:46.606 UTC [1] LOG:  database system is ready to accept connections
2026-09-14 10:38:46.693 UTC [55] LOG:  checkpoint starting: time
2026-09-14 10:38:50.924 UTC [55] LOG:  checkpoint complete...
```

No entry anywhere in the log corresponds to the connection tests performed in Step 2, even though those were genuine TCP connections reaching the database from outside Docker entirely. PostgreSQL does not log connections under its default configuration (`log_connections` is off). Also noted in the startup log:

```
initdb: warning: enabling "trust" authentication for local connections
```

Secondary note: "trust" authentication means some connections are accepted with no password check at all — flagged for follow-up in Session 5 (credentials/encryption), not pursued further here.

Step 4 — Replaying the Session 1 Attack Chain Under Observation

Re-ran the key Session 1 findings while watching each container's logs live, to compare how (or whether) critical events are distinguishable from routine ones.

```
$ curl http://localhost:8080/debug
$ curl http://localhost:8081/backup-notes.txt
$ curl -X POST http://localhost:8080/login -d "username=svc-rep&password=trustno1"
$ curl http://localhost:8080/record/1003
```

web (Flask/Werkzeug) log output

```
* Debug mode: on
* Debugger PIN: 343-425-374
172.19.0.1 - - [14/Sep/2026 10:34:15] "GET /health HTTP/1.1" 200 -
172.19.0.1 - - [14/Sep/2026 11:51:28] "GET /debug HTTP/1.1" 200 -
172.19.0.1 - - [14/Sep/2026 11:51:28] "POST /login HTTP/1.1" 200 -
172.19.0.1 - - [14/Sep/2026 11:51:29] "GET /record/1003 HTTP/1.1" 200 -
```

Two additional issues surfaced incidentally here: **debug mode is on** (planted weakness #7), and the **Flask debugger PIN is printed straight into the container logs**. Since debug mode exposes an interactive in-browser Python console gated by that PIN, having the PIN readable in the logs undermines the one control meant to protect it — a potential path to remote code execution, worth revisiting in Session 8 (OWASP/CIS review).

Core observation: the request that leaked the database password, app secret and every username (**/debug**) is logged in the exact same format as a harmless health check. The admin-privileged login (**/login**) is logged only as "POST /login ... 200" — the log does not capture the username, whether the login succeeded, or what role was granted. A failed login also returns HTTP 200, so **log-only review cannot distinguish a failed login attempt from a successful admin takeover**.

files (nginx) log output

```
172.18.0.1 - - [14/Sep/2026:11:51:28 +0000] "GET /backup-notes.txt HTTP/1.1" 200 209 "-" "curl/8.19.0" "-"
```

nginx captures more fields than Werkzeug (response size, User-Agent), but the same core gap applies: the request for backup-notes.txt — which leaks unencrypted-backup details and a working credential pair — is logged identically to a request for any harmless static file. Nothing marks it as sensitive.

Step 5 — The Two Most Significant Visibility Gaps

Gap 1: No severity or context in application/file-share logging

GAP

Across both web and files, every request — routine or critical — produces the same flat log line: source IP, timestamp, path, status code. There is no distinction between a health check and a full secrets leak. Login outcome (success/failure) and the identity/role involved are not captured in the log at all, only in the HTTP response body, which no log source records. A reviewer reading these logs after an incident would have no way to identify which requests mattered without independently knowing which endpoints are sensitive in advance.

Gap 2: Database access generates no log trace by default

GAP

The database port is reachable directly from the host, bypassing the frontend/backend network split entirely (Step 2). Despite this, PostgreSQL's default configuration does not log connections, successful or otherwise (Step 3). This is the most severe combination in the environment: the weakest access-control boundary is paired with the least visibility of any container — an attacker connecting directly to the database would leave no evidence anywhere in the stack.

Conclusion

PrepLab's only "controls" — Docker network segmentation and per-container logging — are both present but shallow. Segmentation is bypassed by a single unnecessary port mapping, and logging exists everywhere but carries no severity, no authentication context, and in the database's case, no connection record at all. These two gaps carry directly into Session 4 (testing whether the separation actually holds under active probing) and reinforce the Session 2 risk list: the highest-priority risks (credential exposure, IDOR, over-privileged accounts) are also the ones least likely to ever be noticed by anyone watching these logs.